

OpenGL Introduction Session Guide

Fast Realistic Rendering

In this first session, we will apply the OpenGL concepts we have just covered. Although this is an introductory session, we will begin working with the skeleton that we will use throughout the first project.

The tasks for this session are as follows. As you will see, the provided code compiles, but when executed, nothing is displayed.

Render the sphere

Goal: Render a sphere and illuminate it using the Phong shading model. Light must be placed at (1,1,1) in World Space.

1. Complete the `LoadModel` function (lines 148-154 of `glwidget.cc`) to load models into the graphics pipeline.
2. Implement the rendering call in the rendering loop (lines 410-413 of `glwidget.cc`).
3. Implement the Phong shader (`phong.vert` and `phong.frag`) to render the sphere with Phong shading.

Render the skybox

Goal: Render a cube textured with an environment texture to model a skybox that surrounds the sphere.

1. Complete the `LoadModel` function (lines 157-172 of `glwidget.cc`) to load models into the graphics pipeline.
2. Implement the rendering call in the rendering loop (lines 437-439 of `glwidget.cc`).
3. Implement the sky shader (`sky.vert` and `sky.frag`) to render the sky texture on the cube.

Note: The textures are already set up. You only need to access them in the shader using the `texture` function and a 3D vector. To load a skybox, you have an option in the file menu and you have a folder with examples.

Render the sphere textured

Goal: Apply different textures to the sphere to represent its material properties: color, roughness, and metalness.

1. Manage the color, roughness, and metalness textures in the functions:
 - `LoadColorMap` (lines 201-205)
 - `LoadRoughnessMap` (lines 213-217)
 - `LoadMetalnessMap` (lines 224-228)in `glwidget.cc`.

- For reference, check lines 183-197.
- 2. Use these textures in the rendering loop (lines 396-401 of `glwidget.cc`). For reference, check lines 388-394.
- 3. Visualize the textures using the texture shaders (`texMap.vert` and `texMap.frag`).

Note: To load textures, you have an option in the file menu and you have a folder with examples. If you use your own, ensure they have at least three channels (RGB).

Render the skybox reflected on the sphere

Goal: Simulate the sphere's material as a perfect mirror.

1. Implement the reflection shader (`reflection.vert` and `reflection.frag`) following this approach:
 - The view ray hits the sphere and bounces off its surface based on the contact point's normal.
 - The reflected ray then intersects with the skybox.
 - The sphere's surface color is determined by the skybox color at the intersection point.